

GUÍA DE REPASO COMPLETA

Módulo 1 · Unidad 1 · Clase 2

Matemáticas Aplicadas con Python

¿Para quién es esta guía?

Esta guía cubre los dos temas centrales de la Clase 2: álgebra lineal con `numpy.linalg` y la implementación del Descenso del Gradiente en Python. Está diseñada para que puedas repasar, reforzar o ponerte al día sin necesidad de materiales adicionales.

Como en la guía anterior: no es suficiente con leer el código. Si puedes responder '¿por qué se hace esto?' en cada paso, has comprendido el tema.

SECCIÓN 0 | Mapa Conceptual de la Clase

Antes de entrar al detalle, revisa el panorama completo. Esta clase tiene dos bloques matemáticos que se conectan de manera directa: el álgebra lineal es el lenguaje con el que los modelos representan datos, y el descenso del gradiente es el mecanismo con el que los modelos aprenden.

Bloque	Qué cubre y por qué es importante
Bloque A	Álgebra Lineal con <code>numpy.linalg</code> : escalares, vectores, matrices y tensores como estructuras de datos. Operaciones esenciales: norma, determinante, inversa, producto matricial, valores propios, resolución de sistemas lineales. Sin estas operaciones, ningún modelo de ML puede existir matemáticamente.
Bloque B	Descenso del Gradiente: el algoritmo de optimización que usa prácticamente toda la IA moderna. Implementación desde cero en Python con un caso real de predicción de ventas. Entender este algoritmo es entender cómo 'aprende' un modelo.

Conexión entre los dos bloques

Un modelo de Machine Learning recibe los datos como matrices y vectores (Bloque A). Para encontrar los mejores parámetros que minimizan el error de predicción, el modelo aplica el Descenso del Gradiente (Bloque B). Bloque A es la estructura; Bloque B es el motor. Los dos son inseparables en la práctica.

SECCIÓN 1 | Álgebra Lineal con `numpy.linalg`

1.1 Escalares, Vectores, Matrices y Tensores

Antes de escribir cualquier código, necesitas tener claro qué tipo de estructura de datos estás manejando. En álgebra lineal, todos los datos se clasifican según su número de dimensiones:

Concepto	Dimensiones	Ejemplo en contexto bancario	En NumPy
Escalar	0D (un solo número)	La tasa de interés de un crédito: 14.5%	<code>np.array(14.5)</code>
Vector	1D (lista de números)	Los scores de crédito de 5 clientes: [720, 650, 780, 690, 810]	<code>np.array([720, 650, 780, 690, 810])</code>
Matriz	2D (filas y columnas)	Tabla de 3 clientes x 3 variables financieras	<code>np.array([[...], [...], [...]])</code>
Tensor	3D o más	Datos de 3 clientes x 3 variables x 2 meses	<code>np.array</code> con 3 dimensiones

La propiedad `.ndim` de un array de NumPy te dice cuántas dimensiones tiene. La propiedad `.shape` te dice cuántos elementos hay en cada dimensión. Por ejemplo, una matriz de forma `(3, 3)` tiene 3 filas y 3 columnas. Un tensor de forma `(2, 3, 3)` tiene 2 'capas', cada una con 3 filas y 3 columnas.

1.2 Operaciones con `numpy.linalg`

El módulo `numpy.linalg` (`linalg` = linear algebra) contiene las funciones de álgebra lineal de NumPy. No necesitas instalarlo por separado: viene incluido con NumPy. Para usarlo, simplemente escribes `np.linalg.nombre_funcion()`. A continuación se explica cada operación antes de verla en código.

Norma de un vector (`np.linalg.norm`)

La norma de un vector mide su 'longitud' o magnitud en el espacio matemático. La norma más común (norma euclidiana o L2) se calcula como la raíz cuadrada de la suma de los cuadrados de sus componentes. En machine learning, la norma se usa para medir la distancia entre dos puntos, para normalizar vectores, y como parte de funciones de regularización que penalizan modelos demasiado complejos.

Para el vector `[0.3, -0.4, 0.5]`: $\text{norma} = \sqrt{(0.3^2 + (-0.4)^2 + 0.5^2)} = \sqrt{(0.09 + 0.16 + 0.25)} = \sqrt{0.5} \approx 0.7071$

Determinante (`np.linalg.det`)

El determinante es un número que resume propiedades fundamentales de una matriz cuadrada. Si el determinante es cero, la matriz es 'singular' y no tiene inversa (lo que en términos de datos significa que hay variables redundantes o perfectamente correlacionadas). Si

el determinante es distinto de cero, la matriz tiene inversa y el sistema de ecuaciones tiene solución única.

Para la matriz $\begin{bmatrix} 1.0 & 0.6 \\ 0.6 & 1.0 \end{bmatrix}$: $\det = (1.0 \times 1.0) - (0.6 \times 0.6) = 1.0 - 0.36 = 0.64$

Inversa de una matriz (`np.linalg.inv`)

La inversa de una matriz A es otra matriz A^{-1} tal que $A \times A^{-1} = I$, donde I es la matriz identidad (unos en la diagonal, ceros en el resto). Calcular la inversa de una matriz es el equivalente matricial de dividir por un número. Se usa en regresión lineal clásica para encontrar los coeficientes óptimos directamente mediante la ecuación normal.

Producto matricial (operador `@`)

El producto matricial multiplica dos matrices según la regla fila-por-columna. En Python 3, el operador `@` realiza este producto. Es distinto de `*`, que multiplica elemento por elemento. El producto matricial es la operación central de prácticamente todo modelo de ML: al multiplicar la matriz de datos X por el vector de pesos w , obtienes las predicciones del modelo.

Valores y vectores propios (`np.linalg.eig`)

Los valores propios y vectores propios de una matriz revelan sus 'direcciones naturales' de variación. En ciencia de datos, esta operación es la base del Análisis de Componentes Principales (PCA), que se verá en módulos posteriores. Los valores propios indican cuánta varianza explica cada componente. Una propiedad útil para verificar resultados: la suma de los valores propios debe igualar la traza de la matriz (suma de su diagonal), y el producto de los valores propios debe igualar el determinante.

Solución de sistemas lineales (`np.linalg.solve`)

`np.linalg.solve(A, b)` resuelve el sistema de ecuaciones $A \times w = b$, encontrando el vector w que satisface el sistema. Es más eficiente y numéricamente estable que calcular $A^{-1} \times b$. Se usa cuando tienes un sistema de ecuaciones con tantas ecuaciones como incógnitas.

SECCIÓN 2 | Caso Desarrollado: Bancolombia y el Modelo de Riesgo Crediticio

Contexto verificado: Bancolombia

Bancolombia S.A. es el banco más grande de Colombia por activos. Su historia inicia el 29 de enero de 1875 con la fundación del Banco de Colombia en Medellín por Ramón del

Corral. En 1998 se fusionó con el Banco Industrial Colombiano (BIC, fundado en 1945) para formar Bancolombia tal como existe hoy. Opera en Colombia, Panamá, El Salvador, Guatemala, Puerto Rico y las Islas Caimán, con más de 25 millones de clientes según Forbes 2021.

Caso de uso: el equipo de riesgo de Bancolombia evalúa solicitudes de crédito usando modelos matemáticos. Cada cliente tiene variables financieras que se representan como vectores y matrices. Las operaciones de álgebra lineal que aprenderás aquí son la base matemática de esos modelos.

Paso 1: Crear la matriz de datos de clientes

El primer paso es representar los datos de los clientes en la estructura matemática adecuada. Tenemos 3 clientes y 3 variables cada uno: ingreso mensual (millones COP), deuda total (millones COP) y score de historial crediticio (escala 300-850).

Una matriz de 3 filas × 3 columnas representa exactamente esta información. Cada fila es un cliente; cada columna es una variable.

```
import numpy as np

# Matriz de clientes: 3 filas (clientes) x 3 columnas (variables)
# Columna 0: ingreso mensual (millones COP)
# Columna 1: deuda total (millones COP)
# Columna 2: score crediticio (escala 300-850)
clientes = np.array([
    [4.5, 12.0, 720], # Cliente 1
    [2.8, 8.5, 650],  # Cliente 2
    [7.2, 20.0, 780], # Cliente 3
])

print('Matriz de clientes:')
print(clientes)
print('Shape:', clientes.shape)
print('Dimensiones:', clientes.ndim)
print('Tipo de datos:', clientes.dtype)
```

```
Matriz de clientes:
[[ 4.5  12.  720.]
 [ 2.8   8.5 650.]
 [ 7.2  20.  780.]]
Shape: (3, 3)
Dimensiones: 2
```

Tipo de datos: float64

Interpretación: Paso 1

Interpretación técnica: shape (3, 3) confirma que tenemos 3 filas y 3 columnas. NumPy convirtió todos los valores a float64 porque la matriz contiene tanto decimales como enteros: NumPy usa el tipo más amplio para mantener consistencia. ndim = 2 confirma que es una matriz (estructura 2D), no un vector (1D) ni un tensor (3D+).

Interpretación de negocio: cada fila es el 'perfil financiero' de un cliente. El Cliente 3 tiene el mayor ingreso (7.2M) pero también la mayor deuda (20M), y el mejor historial (780). Antes de cualquier modelo, esta representación matricial ya permite comparar clientes visualmente. La lógica de 'cada fila es una observación, cada columna es una variable' es el estándar universal en ciencia de datos.

Paso 2: Tensores — datos a través del tiempo

En la práctica, Bancolombia no evalúa a un cliente una sola vez: hace seguimiento mensual. Si agrego un segundo mes de datos, necesito una estructura 3D: un tensor. El tensor organiza los datos como (número de meses, número de clientes, número de variables).

```
# Tensor 3D: 2 meses x 3 clientes x 3 variables
tensor_meses = np.array([
    [[4.5, 12.0, 720], [2.8, 8.5, 650], [7.2, 20.0, 780]], # Enero
    [[4.8, 11.0, 730], [3.0, 8.0, 660], [7.5, 19.5, 790]], # Febrero
])

print('Shape del tensor:', tensor_meses.shape)
print('Datos de enero (índice 0):')
print(tensor_meses[0])
print('Ingresos de todos los clientes en febrero:', tensor_meses[1, :, 0])
```

```
Shape del tensor: (2, 3, 3)
Datos de enero (índice 0):
[[ 4.5  12.  720.]
 [ 2.8   8.5 650.]
 [ 7.2  20.  780.]]
Ingresos de todos los clientes en febrero: [4.8 3.  7.5]
```

Interpretación: Paso 2

Interpretación técnica: tensor_meses[1, :, 0] usa indexación avanzada: índice 1 selecciona febrero, : selecciona todos los clientes, índice 0 selecciona la primera columna

(ingreso). El resultado [4.8, 3.0, 7.5] son los ingresos de los tres clientes en febrero.

Interpretación de negocio: los tres clientes mejoraron ligeramente entre enero y febrero (ingresos subieron, deudas bajaron o se mantuvieron, scores subieron). Esta estructura de tensor es exactamente cómo TensorFlow y PyTorch almacenan datos para redes neuronales: imágenes (alto x ancho x canales de color), series de tiempo (tiempo x variables), o videos (fotogramas x alto x ancho x canales).

Paso 3: Norma y producto matricial

Ahora aplico las operaciones de `numpy.linalg`. El equipo de riesgo de Bancolombia diseña un vector de pesos que refleja la importancia de cada variable: el ingreso tiene un efecto positivo en la capacidad de pago (+0.3), la deuda tiene un efecto negativo (-0.4), y el historial crediticio es el factor más importante (+0.5).

Con estos pesos, el producto matricial calcula un score de riesgo para cada cliente en una sola operación.

```
# Vector de pesos del modelo de riesgo
# ingreso(+0.3), deuda(-0.4), historial(+0.5)
pesos = np.array([0.3, -0.4, 0.5])

# Norma del vector de pesos
norma = np.linalg.norm(pesos)
print(f'Norma del vector de pesos: {norma:.4f}')

# Producto matricial: score de riesgo por cliente
scores = clientes @ pesos
print('Scores de riesgo por cliente:', scores)
```

```
Norma del vector de pesos: 0.7071
Scores de riesgo por cliente: [356.55 322.44 384.16]
```

Interpretación: Paso 3

Verificación del score del Cliente 1: $4.5 \times 0.3 + 12.0 \times (-0.4) + 720 \times 0.5 = 1.35 - 4.80 + 360.00 = 356.55$. El valor alto de 356.55 se debe principalmente al score crediticio ($720 \times 0.5 = 360$), que domina el resultado por su magnitud.

Interpretación técnica: la norma 0.7071 es la 'longitud' matemática del vector de pesos. El operador `@` realiza el producto matricial en una línea, calculando el score de los tres clientes simultáneamente sin ningún loop.

Interpretación de negocio: el Cliente 3 tiene el mayor score (384.16), lo que indica mayor capacidad crediticia según este modelo. El Cliente 2 tiene el menor score (322.44). Bancolombia usaría estos scores para priorizar aprobaciones y establecer límites de crédito. En la práctica, los pesos del modelo se aprenden de datos históricos, no se asignan manualmente.

Paso 4: Determinante, Inversa y Valores Propios

Para analizar la relación entre variables, el equipo construye una matriz de correlación que muestra qué tan relacionados están el ingreso y la deuda de los clientes. Un valor de 0.6 indica correlación positiva moderada.

```
# Matriz de correlación: ingreso vs deuda
# 1.0 en la diagonal (cada variable está perfectamente correlacionada consigo misma)
# 0.6 fuera de la diagonal (correlación moderada entre ingreso y deuda)
M_corr = np.array([
    [1.0, 0.6],
    [0.6, 1.0]
])

# Determinante
det = np.linalg.det(M_corr)
print(f'Determinante: {det:.4f}')

# Inversa
inv = np.linalg.inv(M_corr)
print('Inversa:')
print(np.round(inv, 4))

# Verificación: M @ M_inv debe dar la identidad
print('Verificación M @ M_inv:')
print(np.round(M_corr @ inv, 10))

# Valores y vectores propios
vals, vecs = np.linalg.eig(M_corr)
print('Valores propios:', np.round(vals, 4))
print('Vectores propios:')
print(np.round(vecs, 4))
```

```
Determinante: 0.6400
```

```
Inversa:
```

```
[[ 1.5625 -0.9375]
```

```

[-0.9375  1.5625]]
Verificación M @ M_inv:
[[ 1. -0.]
 [ 0.  1.]]
Valores propios: [1.6 0.4]
Vectores propios:
[[ 0.7071 -0.7071]
 [ 0.7071  0.7071]]

```

Interpretación: Paso 4

Determinante (0.64): es distinto de cero, lo que confirma que la matriz tiene inversa y que las variables no están perfectamente correlacionadas. Si el determinante fuera 0, significaría que una variable es combinación lineal de la otra (redundancia total).

Inversa: la verificación $M @ M_inv$ da exactamente la identidad $[[1,0],[0,1]]$, confirmando que el cálculo es correcto.

Valores propios [1.6, 0.4]: su suma es 2.0 (igual a la traza de M: $1.0 + 1.0 = 2.0$) y su producto es 0.64 (igual al determinante). Estas son propiedades matemáticas que siempre puedes usar para verificar tus cálculos.

Interpretación de negocio: el valor propio de 1.6 indica que existe una dirección en los datos que explica el 80% de la varianza (1.6 de 2.0 total). En PCA, que verás en módulos posteriores, este análisis determina qué combinación de variables explica más variabilidad en los datos de clientes.

Paso 5: Resolución de un sistema lineal

Supón que el equipo quiere encontrar automáticamente los pesos del modelo para que dos clientes específicos obtengan sus scores objetivo de aprobación (0.85 y 0.62, en escala normalizada). Esto es un sistema de ecuaciones lineales que `numpy.linalg.solve` resuelve directamente.

```

# Sistema de ecuaciones: A @ w = b
# A: datos de 2 clientes (2 variables: ingreso, score)
# b: scores objetivo de aprobación
A = np.array([[4.5, 720],
              [2.8, 650]], dtype=float)
b = np.array([0.85, 0.62])

w = np.linalg.solve(A, b)
print('Pesos encontrados w:', np.round(w, 6))

```

```
# Verificación: A @ w debe reproducir b
print('Verificación A @ w:', np.round(A @ w, 6))
```

```
Pesos encontrados w: [0.116722 0.000451]
Verificación A @ w: [0.85 0.62]
```

Interpretación: Paso 5

Interpretación técnica: `np.linalg.solve` encontró los pesos exactos `[0.116722, 0.000451]` que satisfacen el sistema. La verificación `A @ w` reproduce exactamente `b = [0.85, 0.62]`, confirmando la solución.

Interpretación de negocio: el peso del ingreso (`0.116722`) es mucho más grande que el peso del score (`0.000451`). Esto indica que, en este sistema, el ingreso mensual es la variable con mayor influencia en el score objetivo. Este tipo de análisis es el fundamento matemático de la Regresión Lineal, que verás en detalle en la Unidad 3.

SECCIÓN 3 | Cálculo y Optimización: El Descenso del Gradiente

3.1 ¿Qué es el Descenso del Gradiente y por qué es la base de toda la IA moderna?

El Descenso del Gradiente es el algoritmo de optimización más importante en machine learning. Su función es encontrar los parámetros (pesos) de un modelo que minimizan el error entre las predicciones del modelo y los valores reales.

La metáfora clásica: imagina que estás en una montaña completamente nublada, sin visibilidad. Tu único instrumento es una brújula que te indica la pendiente del terreno bajo tus pies. Para llegar al valle más bajo (mínimo del error), sigues una estrategia simple: en cada paso, muévete en la dirección de mayor descenso según lo que indica la pendiente. Si repites este proceso suficientes veces, eventualmente llegarás al fondo del valle.

Esto es exactamente lo que hace el Descenso del Gradiente con los parámetros del modelo: calcula la 'pendiente' del error (el gradiente) y ajusta los parámetros paso a paso en la dirección que reduce ese error.

3.2 Los componentes del algoritmo

Concepto	Definición	Notación matemática
Función de costo (J)	Mide qué tan equivocadas son las predicciones del modelo. El objetivo es minimizarla. La más común es el Error Cuadrático Medio (MSE).	$J = (1/2m) \times \sum (\hat{y} - y)^2$
Gradiente (∇J)	La derivada parcial de la función de costo con respecto a cada parámetro. Indica la dirección y magnitud del ascenso máximo del error.	$dJ/dw, dJ/db$
Learning rate (α)	Controla qué tan grande es cada paso. Demasiado grande: el modelo oscila o diverge. Demasiado pequeño: el entrenamiento es lento.	Valor típico: 0.001 a 0.1
Iteración	Una actualización de los parámetros. En cada iteración, se calcula el gradiente y se actualiza w y b.	Repetir hasta convergencia

3.3 Las fórmulas de actualización

En cada iteración, los parámetros se actualizan restando el gradiente multiplicado por el learning rate:

```
w_nuevo = w_actual - alpha * (dJ/dw)
b_nuevo = b_actual - alpha * (dJ/db)

# Donde las derivadas parciales son:
dJ/dw = (1/m) * sum((y_predicho - y_real) * X)
dJ/db = (1/m) * sum(y_predicho - y_real)
```

El signo negativo es crucial: restamos el gradiente porque el gradiente apunta hacia el ascenso máximo del error, y nosotros queremos descender. Restar el gradiente nos mueve en la dirección opuesta, es decir, hacia el mínimo del error.

SECCIÓN 4 | Caso Desarrollado: Grupo Éxito — Predicción de Ventas con Descenso del Gradiente

Contexto verificado: Grupo Éxito

Grupo Éxito es el mayor retailer de Colombia. El primer almacén fue abierto el 26 de marzo de 1949 por Gustavo Toro Quintero en el centro de Medellín, entonces un local de aproximadamente 4 m² dedicado a la venta de telas y confecciones. Hoy opera en Colombia, Uruguay y Argentina con marcas como Éxito, Carulla, Surtimax y Surtimayorista. Es referente en el uso de tecnología e inteligencia de datos para gestión de inventarios,

precios dinámicos y predicción de demanda.

Caso de uso: el equipo analítico de Grupo Éxito quiere construir un modelo que prediga las ventas mensuales de una categoría de productos en función de la inversión en publicidad digital. Usaremos el Descenso del Gradiente para que el modelo aprenda la relación entre estas dos variables.

Paso 1: Los datos y el problema de optimización

Tenemos datos históricos de 8 meses: inversión en publicidad digital (millones COP) y ventas resultantes (millones COP). El objetivo es encontrar los parámetros w (pendiente) y b (intercepto) de la línea recta que mejor se ajusta a estos datos.

```
import numpy as np

# Datos históricos: 8 meses
publicidad = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0])
ventas      = np.array([52.3, 55.1, 57.8, 60.2, 62.9, 65.5, 68.0, 70.7])

print('Publicidad (millones COP):', publicidad)
print('Ventas reales (millones COP):', ventas)

# Costo inicial con w=0, b=0 (modelo sin entrenar)
m = len(publicidad)
y_pred_inicial = 0 * publicidad + 0 # prediccion = 0 para todo
costo_inicial = (1 / (2 * m)) * np.sum((y_pred_inicial - ventas) ** 2)
print(f'Costo inicial (modelo sin entrenar): {costo_inicial:.4f}')
```

```
Publicidad (millones COP): [1. 2. 3. 4. 5. 6. 7. 8.]
Ventas reales (millones COP): [52.3 55.1 57.8 60.2 62.9 65.5 68. 70.7]
Costo inicial (modelo sin entrenar): 1912.8331
```

Interpretación: Paso 1

Interpretación técnica: el costo inicial de 1912.83 es el error promedio cuando el modelo no ha aprendido nada ($w=0$, $b=0$, predicción siempre 0). Las ventas reales promedian alrededor de 61.6M, por lo que el error cuadrático es enorme. El descenso del gradiente deberá reducir este costo progresivamente.

Interpretación de negocio: los datos muestran una relación clara: a mayor inversión en publicidad, mayores ventas. Cada millón extra de publicidad parece generar aproximadamente 2.6 millones adicionales en ventas (relación que el modelo aprenderá exactamente).

Paso 2: Implementación del Descenso del Gradiente

Ahora implementamos el algoritmo desde cero. La función encapsula la lógica completa: inicialización de parámetros, loop de iteraciones, cálculo del costo, cálculo de gradientes, y actualización de parámetros.

```
def descenso_gradiente(X, y, alpha=0.01, n_iter=5000):
    m = len(X)          # número de observaciones
    w = 0.0              # pendiente inicial
    b = 0.0              # intercepto inicial
    historia_costo = []  # para registrar el costo en cada iteración

    for i in range(n_iter):
        # 1. Calcular predicciones con los parámetros actuales
        y_pred = w * X + b

        # 2. Calcular el costo (Error Cuadrático Medio)
        costo = (1 / (2 * m)) * np.sum((y_pred - y) ** 2)
        historia_costo.append(costo)

        # 3. Calcular los gradientes
        dw = (1 / m) * np.sum((y_pred - y) * X)
        db = (1 / m) * np.sum(y_pred - y)

        # 4. Actualizar los parámetros
        w = w - alpha * dw
        b = b - alpha * db

    return w, b, historia_costo

w_final, b_final, historia = descenso_gradiente(
    publicidad, ventas, alpha=0.01, n_iter=5000
)

print(f'Parámetro w (pendiente): {w_final:.4f}')
print(f'Parámetro b (intercepto): {b_final:.4f}')
print(f'Ecuación del modelo: ventas = {w_final:.4f} * publicidad + {b_final:.4f}')
```

```
Parámetro w (pendiente): 2.6087
Parámetro b (intercepto): 49.8228
Ecuación del modelo: ventas = 2.6087 * publicidad + 49.8228
```

Paso 3: Verificar la convergencia

El descenso del gradiente funciona si el costo disminuye progresivamente con cada iteración. Verifiquemos el costo en momentos clave del entrenamiento:

```
print('Evolución del costo durante el entrenamiento:')
print(f' Iteración      0: costo = {historia[0]:.4f}')
```

```
print(f' Iteración      1: costo = {historia[1]:.4f}')
```

```
print(f' Iteración     10: costo = {historia[10]:.4f}')
```

```
print(f' Iteración    100: costo = {historia[100]:.4f}')
```

```
print(f' Iteración   500: costo = {historia[500]:.4f}')
```

```
print(f' Iteración  1000: costo = {historia[1000]:.4f}')
```

```
print(f' Iteración  4999: costo = {historia[4999]:.4f}')
```

Evolución del costo durante el entrenamiento:

```
Iteración      0: costo = 1912.8331
Iteración      1: costo = 1145.7313
Iteración     10: costo = 230.2449
Iteración    100: costo = 158.0744
Iteración   500: costo = 31.9644
Iteración  1000: costo = 4.3369
Iteración  4999: costo = 0.0035
```

Interpretación: Paso 3

Interpretación técnica: el costo cayó de 1912.83 a 0.0035, una reducción del 99.99%. El descenso fue más rápido al principio (el gradiente era grande cuando el error era enorme) y se fue desacelerando a medida que el modelo se aproximaba al mínimo. Este comportamiento es característico del descenso del gradiente y se conoce como 'aprendizaje decreciente'.

Nota importante: se necesitaron 5000 iteraciones para converger porque los datos no están normalizados (publicidad en escala 1-8, ventas en escala 52-71). Con datos normalizados, la convergencia puede ser cien veces más rápida. La normalización de datos antes del entrenamiento es una buena práctica que estudiarás en profundidad en las unidades siguientes.

Paso 4: Usar el modelo para predecir

```
# Predicciones con el modelo entrenado
pred_2M  = w_final * 2.0 + b_final
pred_5M  = w_final * 5.0 + b_final
pred_10M = w_final * 10.0 + b_final
```

```
print(f'Publicidad 2M COP → ventas estimadas: {pred_2M:.2f}M COP')
print(f'Publicidad 5M COP → ventas estimadas: {pred_5M:.2f}M COP')
print(f'Publicidad 10M COP → ventas estimadas: {pred_10M:.2f}M COP')

# Verificación con np.polyfit (mínimos cuadrados exactos)
coef = np.polyfit(publicidad, ventas, 1)
print(f'\nVerificación np.polyfit: w={coef[0]:.4f}, b={coef[1]:.4f}')
print('El modelo aprendido por GD coincide con la solución exacta.')
```

```
Publicidad 2M COP → ventas estimadas: 55.04M COP
Publicidad 5M COP → ventas estimadas: 62.87M COP
Publicidad 10M COP → ventas estimadas: 75.91M COP
```

```
Verificación np.polyfit: w=2.6083, b=49.8250
El modelo aprendido por GD coincide con la solución exacta.
```

Interpretación: Paso 4

Interpretación técnica: el Descenso del Gradiente ($w=2.6087$, $b=49.8228$) convergió prácticamente al mismo resultado que `np.polyfit` ($w=2.6083$, $b=49.8250$), que calcula la solución exacta por mínimos cuadrados. La diferencia es de 0.0004, negligible. Esto confirma que el algoritmo funcionó correctamente.

Interpretación de negocio: el modelo aprendió que cada millón COP adicional invertido en publicidad digital genera aproximadamente 2.61 millones COP en ventas adicionales. La base de ventas sin publicidad sería de 49.82 millones COP. Con una inversión de 10M COP en publicidad, el modelo predice ventas de 75.91M COP. Grupo Éxito puede usar este modelo para optimizar su presupuesto de marketing: el ROI es 2.61:1 (por cada peso invertido, retornan 2.61 en ventas).

SECCIÓN 5 | Cheat Sheet: Términos, Funciones y Fórmulas Clave

Funciones de `numpy.linalg`

Función / Término	Definición y uso
<code>np.linalg.norm(v)</code>	Calcula la norma (magnitud) de un vector. Por defecto, norma euclidiana L_2 = raíz cuadrada de la suma de los cuadrados.
<code>np.linalg.det(A)</code>	Determinante de una matriz cuadrada A. Si $\det = 0$: matriz singular (sin inversa). Si $\det \neq 0$: la matriz tiene inversa.

<code>np.linalg.inv(A)</code>	Inversa de la matriz A. Tal que $A @ A_inv = \text{Identidad}$. Solo existe si $\det(A) \neq 0$.
<code>np.linalg.eig(A)</code>	Valores propios y vectores propios de A. Devuelve una tupla (valores, vectores). La suma de valores propios = $\text{traza}(A)$; el producto = $\det(A)$.
<code>np.linalg.solve(A, b)</code>	Resuelve el sistema $A @ w = b$. Más eficiente y estable numéricamente que calcular $\text{inv}(A) @ b$.
Operador <code>@</code>	Producto matricial (matriz-por-matriz o matriz-por-vector). Distinto de <code>*</code> , que multiplica elemento por elemento.
<code>array.ndim</code>	Número de dimensiones del array. Escalar=0, vector=1, matriz=2, tensor=3+.
<code>array.shape</code>	Tupla con el número de elementos por dimensión. (3, 4) = 3 filas, 4 columnas.

Conceptos del Descenso del Gradiente

Concepto	Definición y uso
Función de costo (J)	Mide el error del modelo. $MSE = (1/2m) \times \sum (y_pred - y_real)^2$. Objetivo: minimizarla. Cuanto menor el costo, mejores las predicciones.
Gradiente	Derivada parcial de J respecto a cada parámetro. Indica la dirección de mayor ascenso del error. Se resta para descender.
Learning rate (α)	Tamaño de cada paso de actualización. Muy grande: inestabilidad. Muy pequeño: convergencia lenta. Valor típico: 0.001 a 0.1.
Iteración / Época	Una actualización completa de todos los parámetros del modelo. Se repite hasta convergencia (costo estable).
Convergencia	El momento en que el costo deja de disminuir significativamente entre iteraciones. El modelo ha 'aprendido'.
w (pendiente)	Parámetro que modela el efecto de X sobre y. Cuánto cambia y por cada unidad de X.
b (intercepto)	Valor base de y cuando $X = 0$. El punto donde la recta corta el eje vertical.
MSE	Mean Squared Error. Error Cuadrático Medio. Promedio de los errores al cuadrado. Se eleva al cuadrado para penalizar errores grandes y para que la función sea diferenciable.

Fórmulas de referencia rápida

Operación	Fórmula matemática	NumPy / Python
Norma L2	$\sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$	<code>np.linalg.norm(v)</code>
Determinante 2x2	$(a \times d) - (b \times c)$	<code>np.linalg.det(A)</code>

Producto matricial	$\sum(\text{fila}_i \times \text{col}_j)$	$A @ B$
Función de costo MSE	$(1/2m) \times \sum(\hat{y} - y)^2$	$(1/(2*m)) * \text{np.sum}((y_pred-y)**2)$
Gradiente respecto a w	$(1/m) \times \sum(\hat{y} - y) \times x$	$(1/m) * \text{np.sum}((y_pred-y)*X)$
Gradiente respecto a b	$(1/m) \times \sum(\hat{y} - y)$	$(1/m) * \text{np.sum}(y_pred-y)$
Actualización de w	$w = w - \alpha \times (\partial J / \partial w)$	$w -= \text{alpha} * dw$
Actualización de b	$b = b - \alpha \times (\partial J / \partial b)$	$b -= \text{alpha} * db$

SECCIÓN 6 | Ejercicio Propuesto Autónomo

Contexto: Claro Colombia — Optimización de inversión en retención de clientes

Claro Colombia es la operadora de telecomunicaciones con mayor participación de mercado en el país, con presencia en telefonía móvil, internet, televisión y servicios empresariales. Opera en Colombia como filial de América Móvil, el grupo de telecomunicaciones más grande de América Latina, fundado por Carlos Slim. Claro inició operaciones en Colombia en 2012 tras la compra de Telmex Colombia y Comcel.

Caso de uso: el equipo de analítica de Claro Colombia analiza la relación entre la inversión mensual en programas de retención de clientes (millones COP) y el número de clientes que no cancelan su servicio ese mes (miles de clientes retenidos). Quieren construir un modelo predictivo para optimizar el presupuesto.

Datos históricos de 6 meses:

Mes	Inversión en retención (millones COP)	Clientes retenidos (miles)
Mes 1	3.0	210
Mes 2	5.0	230
Mes 3	7.0	248
Mes 4	9.0	265
Mes 5	11.0	283
Mes 6	13.0	299

Instrucciones

Resuelve cada punto en tu Jupyter Notebook. Cada bloque de código debe ir acompañado de una celda Markdown con tu interpretación del resultado en términos del negocio de Claro Colombia.

Punto 1 — Álgebra lineal básica

- Crea el vector de inversión y el vector de clientes retenidos como arrays de NumPy.
- Construye una matriz X de 6×2 que combine: columna 0 = inversión, columna 1 = un vector de unos (`np.ones(6)`). Esta es la estructura estándar para regresión lineal con intercepto.
- Calcula e imprime: la norma del vector de inversión, el producto $X.T @ X$ (X transpuesto por X), y el determinante de $X.T @ X$.
- En tu celda Markdown: ¿qué significa que el determinante de $X.T @ X$ sea distinto de cero en el contexto de un modelo de regresión?

Punto 2 — Solución exacta con `np.linalg.solve`

- Usa `np.linalg.solve` para resolver el sistema $(X.T @ X) @ w = X.T @ y$, donde y es el vector de clientes retenidos. Este sistema es la 'ecuación normal' de la regresión lineal.
- Imprime los parámetros encontrados $w[0]$ (pendiente) y $w[1]$ (intercepto).
- En tu celda Markdown: interpreta los parámetros. ¿Cuántos miles de clientes adicionales se retienen por cada millón COP extra invertido? ¿Cuál sería la retención base sin inversión?

Punto 3 — Descenso del Gradiente

- Implementa el Descenso del Gradiente desde cero para encontrar los mismos parámetros. Usa `alpha=0.001` y `n_iter=5000`.
- Imprime el costo en las iteraciones 0, 100, 500, 1000, y 4999.
- Imprime los parámetros w y b al final del entrenamiento.
- Compara los resultados del Descenso del Gradiente con la solución exacta del Punto 2. ¿Son similares? ¿Qué explica cualquier diferencia?

Punto 4 — Predicciones y análisis de negocio

- Usa el modelo del Punto 3 para predecir: ¿cuántos miles de clientes retendría Claro con una inversión de 15M COP? ¿Y con 20M COP?
- ¿Tiene sentido predecir para inversiones muy grandes (por ejemplo, 100M COP)? ¿Por qué sí o no?
- En tu celda Markdown: calcula el ROI aproximado del programa de retención. Si retener un cliente genera en promedio 80,000 COP en ingresos mensuales, ¿es rentable el programa?

Punto 5 — Reflexión

- ¿En qué se diferencia el enfoque del Punto 2 (`np.linalg.solve`) del Punto 3 (Descenso del Gradiente)? ¿Cuándo usarías cada uno?
- El Descenso del Gradiente es la base del entrenamiento de redes neuronales (Deep Learning). ¿Qué elementos del algoritmo que implementaste en este ejercicio reconocerías si vieras el código de entrenamiento de una red neuronal en TensorFlow?

Criterio de autoevaluación

Tu trabajo está completo si: (1) los parámetros del Punto 2 y del Punto 3 coinciden con una diferencia menor a 0.01; (2) cada resultado numérico tiene una interpretación en lenguaje de negocio, no solo técnica; (3) la reflexión del Punto 5 explica claramente en qué escenarios usarías `np.linalg.solve` versus el Descenso del Gradiente.

Pista para verificar tu resultado del Punto 2: el parámetro `w[0]` (clientes retenidos por millón COP invertido) debe ser aproximadamente 7.4 miles de clientes. Si obtienes un valor muy diferente, revisa la construcción de tu matriz `X`.